

The Shell on the Panel, and Not a Menu of It

UM-NATOS-026

DOCUMENT	REVISION	STATUS	CLASSIFICATION
UM-NATOS-026	1.1 · 2026-08-16	**Complete, verified on hardware** — §4.1 added, scrollbar	Internal

1. Abstract

The shell has always been a front end that happened to read a serial port. This gives it a second front end and changes nothing behind it.

The consequence is larger than the feature: **the device no longer needs a computer attached to be inspected.** `mem`, `ps`, `stacks`, `adc`, `i2c`, `intr`, `calshow` and `kill` were reachable only from a host over USB. The launcher exists precisely to avoid needing a host (UM-NATOS-021 §1), and the shell was the largest remaining reason to plug one in.

It replaces the `counter` icon, which demonstrated that bytecode could increment a number and had done so for some time.

2. The shortcut not taken

The easy version is an on-screen menu of the popular commands — a grid of buttons labelled `mem`, `ps`, `adc`, each calling the function directly. It needs no keyboard, no line buffer and no output capture, and it would have taken an hour.

It would also have created a **second command set**. Every command added to the real shell would be absent from the panel until someone remembered; every command whose output changed would have a stale twin. The two would diverge in the direction of whichever was used more, and nothing would report the drift.

So a line typed on the glass goes through `shell_run_line()` into the same `execute()` a serial line reaches — same parsing, same splitting, same output, same unknown-command message. There is no on-panel command table, because there is no on-panel command set. `help` on the glass lists what `help` lists, because it is `help`.

The one concession is `shell_run_line()` itself: `execute()` splits its argument in place, so a caller's string cannot be passed straight through. The copy is also the length check — a line longer than the shell's buffer is **refused rather than truncated**, because a truncated command is a different command, and `kill 12` truncated to `kill 1` kills something.

3. Output capture

The shell writes to the UART, because that is where a shell writes. Rather than teach it a second destination, `uart_putc()` grew an optional tee and the app installs itself as one.

The tee is installed for exactly the duration of one command and removed immediately. Left on, it would capture the reporter task's telemetry — several lines a second — and the pane would fill with output nobody asked for within about two seconds. `execute()` holds the console lock for its whole run, so while the tee is set the only writer is the command itself.

The capture function runs inside `uart_putc()`. It must not print, must not lock, and must not be slow, since it is called once per character of output.

4. Layout

The same 224-row region the launcher and the note pad own, split three ways. The keyboard is the fixed cost — four rows of 26 px is 104 rows, nearly half the region — and everything else fits in what is left.

band	rows	
header	22	title and the close button
output	87	9 lines at 9 px
input	11	prompt, line, block cursor
keyboard	104	12 keys at 80 × 26

Three `_Static_assert` s hold the arithmetic: the keyboard must end exactly at the region boundary, the output pane must meet the input line, and an output pane under six lines is refused at compile time rather than shipped as a cramped one. That is the same discipline as UM-NATOS-021 §7, arrived at the same way — by having previously broken a layout and not noticed.

The output pane is a **ring of fixed-width lines**, not a byte stream. A stream would need re-wrapping on every draw, and the wrap is already decided at capture time by where the newlines fall.

4.1 Scrollback

Nine visible lines was the wrong size for the two commands most worth running. `help` produces about thirty lines and `adc` about fifteen, so the pane showed the end of a list whose beginning was the part being asked for.

The ring now holds **48 lines** — a little over three screens of `help` — while still showing nine. The bound is deliberate rather than generous: this is a fixed allocation in a kernel with no paging, so scrollback that grew with output would be an unbounded allocation driven by whatever the user typed. 48 lines costs under 2 KB of `.bss`.

Tapping the output pane scrolls it: upper half back, lower half forward, half a screen per tap. A line per tap would be nine taps to move one screen, which on a panel this slow is not a control but a chore; half-screens also leave two lines of overlap so the reader need not remember what the last line was.

A scrollbar appears on the right only when there is something to scroll. It is the affordance as much as the indicator — nothing else on screen says the pane can be scrolled, and a gesture with no visible cue is a feature only its author knows about.

Lines are addressed by **distance back from the write cursor**, never forward from an origin. A ring has no origin once it has wrapped, and counting forward from one means recomputing where it moved to on every

draw. The arithmetic was checked by simulating 70 lines through the 48-line ring and comparing the window against a plain list at all 40 scroll positions.

Submitting a command snaps to the newest; scrolling stays put while reading. Output only ever arrives because a command was submitted, so a pane that jumped mid-read would be worse than one that does not follow.

5. The keypad is a second copy

`term.c` carries its own multi-tap cycling logic, duplicating the note pad's.

This is recorded rather than pretended away. Factoring it into a shared widget means modifying a working application to serve a new one, and the note pad is the app holding the user's saved messages. Two copies with a comment was judged the better trade against changing working code late at night; **if a third consumer appears, that is when it should be extracted**, because at three the duplication stops being a shortcut and becomes a pattern.

The keys are 80×26 , matching the note pad. There, that size was forced — 24 px keys were unusable against the touch error of UM-NATOS-022 §3.2. That error has since been corrected (UM-NATOS-017 §7.4), so here the size is **a choice rather than a workaround**, and the reason is different: shell commands are short lowercase words, and multi-tap types those adequately.

The bottom row differs from the note pad's. There is no `save`; the third key is `run`, because a shell's terminating key submits.

6. Verification

```
double-tap the shell icon
type a command on the keypad, press run
-> the command echoes after a "> " prompt
-> its output appears in the pane
-> the same output appears on the serial port, unchanged
close returns to the launcher
```

The serial half of that matters: it confirms the **tee copies** output rather than diverting it, so attaching a host does not change what the panel shows.

7. Metrics

Quantity	Value
Output pane	9 lines × 39 columns
Input line	40 characters, refused beyond
Keys	12, 80 × 26
Multi-tap commit	80 ticks (~800 ms)
Commands reachable without a host	all 25
Command sets	1
Copies of the keypad in the tree	2

8. What this does not establish

- **48 lines is still a bound.** `help` is about thirty lines, so a `help` followed by anything long pushes its start out of the ring. What leaves is gone — there is no paging to flash and no way to look further back.
- **Scrolling is by tap, so it costs presses.** Reaching the top of a full ring is nine taps.
- **No history, no editing.** `del` removes the last character. There is no cursor movement, no recall of the previous command, and re-running a fifteen-tap command means fifteen taps again.
- **No arguments are convenient.** `sdread 240` is typeable but tedious, and `run gfxrogue` more so. The commands worth using here are the short ones.
- **Uppercase is impossible.** The keypad emits lowercase only. No current command needs otherwise, and nothing enforces that it stays true.
- **The tee is single-slot and not reentrant.** One installer at a time, no nesting, and a command that somehow ran the shell again would lose its own capture.
- **Output during a command from ANOTHER task is captured too,** if that task prints without taking the console lock. Nothing in the tree does, so this has never happened — which is not the same as being prevented.

9. References

- UM-NATOS-021 §1 — the launcher's purpose, which this completes
- UM-NATOS-022 §3 — the multi-tap keypad this duplicates, and why it exists
- UM-NATOS-017 §7.4 — the calibration fix that made 80 px keys a choice
- `kernel/term.c` — the app; `kernel/shell.c` — `shell_run_line()`; `kernel/uart.c` — the tee